

陳定宇 E94141185

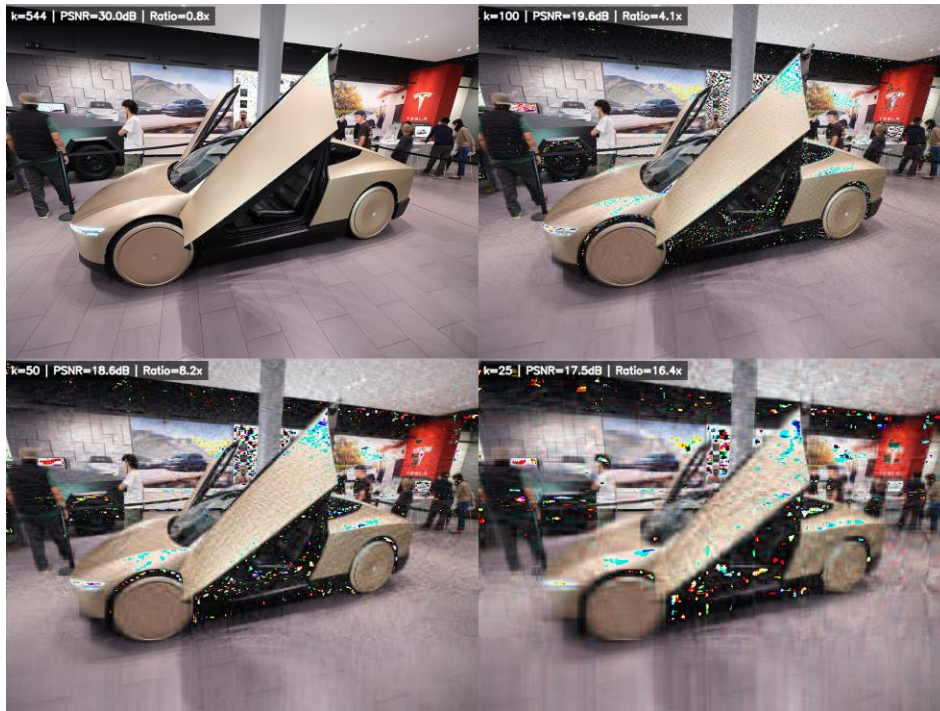
原始圖片相關數據：

原始解析度：960 × 720 像素

通道數：3（BGR 彩色圖片）

矩陣大小：720 × 960 × 3





保留全部奇異值 (**k=544**) :

圖片的整體輪廓、色彩細節皆與原圖一致，非常清晰，車身的光澤、背景人物以及字體邊緣都完美保留，沒有肉眼可見的模糊或失真。

k = 100 :

車身的主要輪廓、金屬光澤和背景大面積的色彩都還算清晰可辨。但仔細觀察會發現，車門邊緣的線條開始出現輕微的模糊，背景人物的面部與衣物等細小紋理已經流失，且畫面中開始出現一些微小的雜訊斑點。

k = 50 :

圖片的清晰度大幅下降。車身表面原本平滑的金屬質感消失，取而代之的是明顯的波浪狀與塊狀（blocky）色塊紋理。背景的 Tesla 招牌字體與展示牆的海報細節已經無法辨識，畫面呈現出一種類似水彩暈開的強烈模糊感，雜訊的干擾也變得更加明顯。

k = 25 :

圖片品質嚴重劣化。雖然勉強還能看出車子的基本構圖與主色調，但所有的表面細節皆已喪失。畫面被大量且密集的塊狀像素與條紋狀失真覆蓋，原本平滑的漸層完全變成了生硬的色塊，色彩也出現了嚴重的雜訊干擾。

加分項與延伸內容說明：

在本次作業中，我不僅完成了基本的 **SVD** 壓縮與顯示，還針對影像的「品質評估」、「壓縮率分析」以及「互動式 UI 介面」進行了額外的延伸實作。詳細說明如下：

1. 影像品質定量評估 (**MSE** 與 **PSNR**) 與 資料壓縮率分析 為客觀評估不同 k 值下的影像失真程度，我引入了 **MSE** (均方誤差) 與 **PSNR** (峰值信噪比) 兩種影像品質指標進行定量分析。 **MSE**：數值越小，代表壓縮後的影像與原圖差異越小。 **PSNR**：數值越大，代表影像品質越好（通常大於 30 dB 代表肉眼難以分辨差異）。同時，我也計算了原圖與壓縮後的資料量差異。原圖資料量為 寬 $\times$ 高 $\times$ 3 個通道；壓縮後所需的資料量則是儲存 **SVD** 三個矩陣所需的總和，公式為 $k \times (\text{寬} + \text{高} + 1) \times 3$ 個通道。

```

=====
SVD 圖像壓縮分析結果
=====

--- k = 544 ---
MSE (均方誤差):      65.2495
PSNR (峰值信噪比):   29.99 dB
原始資料量:          2,073,600 個數值
壓縮後資料量:        2,743,392 個數值
壓縮比:              0.76x
壓縮率:              -32.3%

--- k = 100 ---
MSE (均方誤差):      709.9112
PSNR (峰值信噪比):   19.62 dB
原始資料量:          2,073,600 個數值
壓縮後資料量:        504,300 個數值
壓縮比:              4.11x
壓縮率:              75.7%

--- k = 50 ---
MSE (均方誤差):      897.5887
PSNR (峰值信噪比):   18.60 dB
原始資料量:          2,073,600 個數值
壓縮後資料量:        252,150 個數值
壓縮比:              8.22x
壓縮率:              87.8%

--- k = 25 ---
MSE (均方誤差):      1153.3553
PSNR (峰值信噪比):   17.51 dB
原始資料量:          2,073,600 個數值
壓縮後資料量:        126,075 個數值
壓縮比:              16.45x
壓縮率:              93.9%

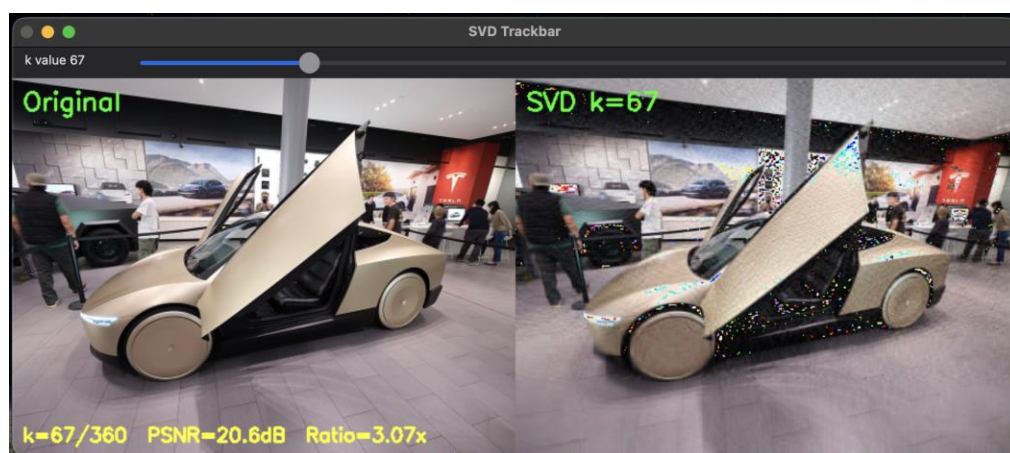
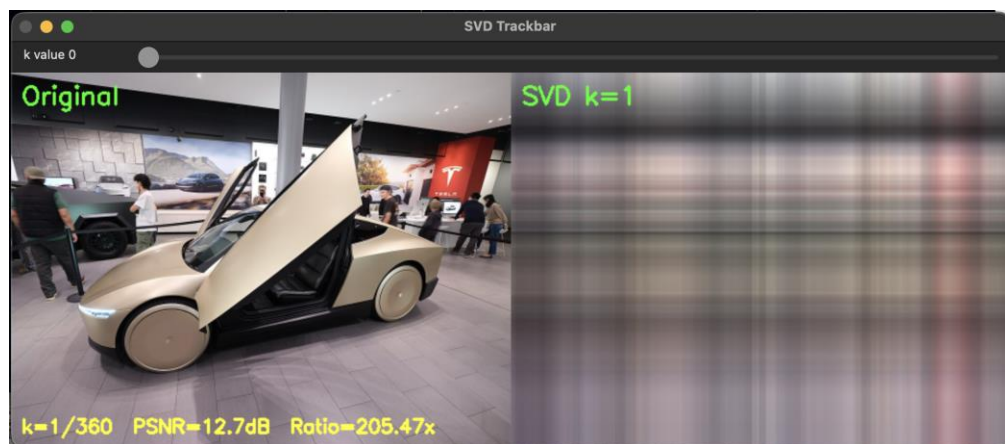
```

=====					
各 k 值壓縮比較表					
=====					
k 值	MSE	PSNR (dB)	壓縮比	壓縮率	

544	65.2495	29.99	0.76	-32.3	%
100	709.9112	19.62	4.11	75.7	%
50	897.5887	18.60	8.22	87.8	%
25	1153.3553	17.51	16.45	93.9	%

2. 動態介面擴充 (OpenCV Trackbar) 為了更直觀地觀察 k 值對影像的影響，我利用 OpenCV 內建的 Trackbar 功能實作了互動式滑桿。在程式的設計上，我特別將運算拆分，先在初始階段將原圖的 SVD 分解計算完畢並暫存。之後每次滑動 Trackbar 時，系統只需取出相對應數量的奇異值矩陣進

行內積重組，如此一來便能達成極為順暢、不卡頓的動態即時預覽效果，讓使用者能清楚看見 k 值增減瞬間對畫質與細節造成的漸進變化。



心得與討論：

這次作業讓我真正體會到線性代數中「奇異值分解 (SVD)」的實用性。以前在課堂上學矩陣分解，總覺得只是在算一堆數學數字，但這次把它實際應用在圖片壓縮上，看到只要保留少數幾個比較大的奇異值（例如 $k=100$ ），就能還原出幾乎跟原圖一樣的車子輪廓，覺得非常神奇，也很有成就感。

為了讓壓縮的效果有比較具體的比較，我不只用肉眼觀察，還去查了怎麼用數字來計算圖片的失真程度，在程式裡多加了 MSE（均方誤差）和 PSNR 這兩個指標，並且算了實際的壓縮比。從我整理出的數據來看，當 $k=100$ 的

時候，其實圖片需要的資料量已經被壓縮了4倍多，但看過去車子的金屬光澤和細節都還在。這證明了 **SVD** 真的很擅長抓出圖片裡的「重點特徵」，把不重要的細節丟掉來節省儲存空間。

另外，我覺得如果每次都要改程式碼裡的 **k** 值再重新執行才能看結果，有點太麻煩了。所以我就去查了 **OpenCV** 的用法，多寫了一個帶有滑動桿

（**Trackbar**）的互動視窗。實作的過程中我學到了一個小技巧：如果每次滑動都重新算一次 **SVD** 電腦會卡住，所以我讓程式先算好 **SVD** 存起來，滑動時只要抓需要的數量出來拼湊就好。這樣一來，我就可以很順暢地左右拉動滑桿，看著圖片從一堆模糊的色塊瞬間變清晰，這個動態變化的過程，讓我對 **SVD** 的實際運作方式有了更深刻的印象！